

Hyderabad Traffic-Focused Agentic City Planner — Master Blueprint

Scope: Hyderabad only. Traffic congestion is the primary domain; pollution and energy are supporting domains linked to traffic. Dual login (citizen / city planner). Constraint-aware agent that recommends rerouting, signal retiming, road widening, flyovers, or drainage upgrades, and tags the responsible Hyderabad authority.

1. Architecture

1.1 Layers

Layer 1 — Entry points

- **Citizen app:** submits complaints as text, image, or video, optionally pinned to a map location.
- **City planner app:** logs in with a `ward_id` claim (assigned at onboarding, matching GHMC's 150 ward divisions) or searches any Hyderabad locality.

Layer 2 — API & Auth gateway

- Validates JWT, resolves role (`citizen` / `planner`).
- For planners, resolves their ward's PostGIS polygon so dashboard queries are scoped by default (read access to other wards is allowed but flagged, since congestion corridors often cross ward boundaries).

Layer 3 — Agentic core (RAG + LLM)

- Retrieves context from the Hyderabad data layer + recent citizen complaints for a locality.
- Surfaces the top 3 problems per domain (traffic first).
- Parses any planner-stated constraints (budget, spatial, timeline, "no road closures during exams," etc.) as **hard filters**, not just prompt context.
- Calls the **rerouting & infrastructure recommendation engine** (§1.4) for traffic fixes, and domain modules for pollution/energy.
- Outputs a recommendation tagged with the responsible authority.

Layer 4 — Hyderabad data layer

- All ingestion is scoped to Hyderabad's bounding box (~17.20°N–17.60°N, 78.25°E–78.65°E) to keep the RAG index precise and prevent the agent pulling in irrelevant-city data from nationwide sources (CPCB, data.gov.in).

Layer 5 — Outputs

- Planner dashboard: overview card, domain tabs, top-3 problems, ranked recommendations.
- Citizen: acknowledgement + tracking ID + (optionally) status updates.
- Authority routing record: locality, issue, recommended fix, cost estimate, responsible department.

1.2 Citizen complaint sub-flow

Text/image/video → (if video: FFmpeg frame extraction + Whisper transcription) → multimodal LLM classifies domain + severity + location → PostGIS matches to a GHMC ward → alert + AI-drafted first-pass fix pushed to the assigned planner (WebSocket + SMS/email) → citizen gets a tracking ID.

1.3 Planner dashboard sub-flow

Login/search locality → overview card (traffic state, infrastructure notes, air/water/land pollution snapshot) → Traffic tab (default) shows top-3 problems with cited evidence → planner can accept a suggestion or re-prompt the agent with new constraints → approved actions are logged against the KPI they're meant to move, closing the feedback loop.

1.4 Rerouting & infrastructure recommendation engine

This is the model layer that turns a diagnosed traffic problem into a specific, costed, feasible fix. Four stages:

1. **Diagnose root cause** (rules + statistics, not an LLM call, for auditability): recurring peak-hour bottleneck, capacity-limited corridor, waterlogging-induced, signal mistiming, or one-off incident.
2. **Generate candidates**, one model per intervention type:
 - *Rerouting* — congestion-weighted shortest path (NetworkX/OSMnx) on live travel-time edge weights, then a SUMO microsimulation checks the diversion doesn't just move the jam elsewhere.
 - *Signal retiming* — Webster's formula for an explainable baseline, refined with an RL agent (SUMO-RL) only where there's enough historical data to trust it.
 - *Road widening* — traffic growth forecast (Prophet) against current capacity, filtered by available right-of-way width from HMDA/GHMC GIS.
 - *Flyover* — only surfaced when volume already exceeds capacity **and** widening fails the ROW feasibility check **and** delay is high outside peak hours too.
 - *Drainage/sewage upgrade* — rational-method sizing ($Q = CiA$) against IMD rainfall data and existing GHMC drain capacity, for waterlogging-driven congestion.
3. **Constraint filter**: each candidate is checked against budget (via a PWD/GHMC schedule-of-rates cost estimator), spatial feasibility (ROW, heritage/no-construction zones), and any free-text constraints the LLM applies directly.

4. **Rank and recommend:** surviving candidates are ranked by estimated congestion reduction per rupee spent; the LLM writes the final recommendation in plain language, citing the evidence, and tags the responsible authority.
-

2. Tech Stack (with justification)

Layer	Choice	Why
LLM / agent reasoning	Claude (Anthropic API), multimodal, tool use	Needs to read text + images natively, reason over hard constraints, and call tools — genuine agentic tool-use, not a simple classifier
Video handling	FFmpeg (frame extraction) + Whisper (transcription), frames + transcript sent to Claude	Claude doesn't take raw video; sampling frames keeps token cost down versus sending full video
RAG / retrieval	LlamaIndex or LangChain + pgvector	pgvector runs inside the same Postgres instance already needed for PostGIS — avoids standing up a second database
Geospatial matching	PostgreSQL + PostGIS	Ward-to-complaint matching is a point-in-polygon query; PostGIS is the standard, well-optimized tool for this
Rerouting graph	NetworkX / OSMnx on the Hyderabad-clipped road network	Free, detailed street-level graph; congestion-weighted shortest path is fast and explainable to a planner
Reroute/candidate validation	SUMO (microsimulation)	The only practical way to check a proposed reroute doesn't create a new bottleneck before recommending it
Signal optimization	Webster's formula baseline + SUMO-RL (TraCI) refinement	Explainable first, black-box only layered in where data supports it — avoids an untrustworthy cold-start recommendation
Growth forecasting	Prophet (or comparable time-series model)	Simple, robust to gaps in historical traffic-volume data, easy to explain to non-technical planners
Time-series sensor data	TimescaleDB extension on the same Postgres instance	Traffic/AQI/energy readings are time-series; avoids a third database
Backend API	FastAPI (Python)	Same language as the ML/agent stack, async-native for concurrent complaint uploads and LLM calls

Layer	Choice	Why
Object storage	S3-compatible storage (MinIO self-hosted, or AWS S3)	Images/videos don't belong in Postgres; keeps the DB lean
Async task queue	Celery + Redis	Video processing and LLM calls shouldn't block the request cycle; complaints get an instant "received" response
Real-time alerts	WebSockets (in-app) + Twilio/SMS or email (offline planners)	Push over poll for near-instant planner awareness of new complaints
Frontend	React + Next.js, Mapbox/Leaflet with GHMC ward GeoJSON overlay	Map-centric UI is core to locality search and the overview card
Auth	JWT + RBAC , planner tokens carry a <code>ward_id</code> claim	Stateless, and encodes the "planner owns a ward" business rule directly in the token
Deployment	Docker + Kubernetes , scheduled ingestion via Airflow	Data ingestion (traffic/AQI/energy pulls) runs on a schedule independent of user traffic
Monitoring	Prometheus + Grafana	Standard, free; doubles as the KPI dashboard for tracking congestion-index trends over time

3. Workflow

A. Citizen complaint

1. Login → submit text/image/video, optionally pin location.
2. Video → FFmpeg frame extraction + Whisper transcription; frames/transcript/text sent to the multimodal LLM.
3. LLM classifies domain, severity, and (if not pinned) infers or requests location.
4. PostGIS matches coordinate to a GHMC ward → identifies the assigned planner.
5. Agent drafts a first-pass fix using the same RAG context planners see; alert pushed to the planner in real time.
6. Citizen gets a tracking ID.

B. City planner

1. Login → lands on assigned ward, or searches another locality (Hyderabad-only geocoding).
 2. Overview card renders: traffic state, infrastructure notes, air/water/land pollution snapshot.
 3. Traffic tab (default) → top-3 problems, each with cited data + linked citizen complaints.
 4. Planner asks the agent for a fix, optionally stating constraints (budget, spatial limits, timeline, other) → constraint parser extracts these as hard filters → rerouting/infrastructure engine (§1.4) runs → ranked, costed recommendation returned with the responsible authority tagged.
 5. Planner can request a revision under different constraints, or switch to Pollution/Energy tabs (each shows how a traffic fix affects them, e.g. "reduces roadside PM2.5 by ~X%").
 6. Approved action is logged and tracked against its target KPI — this outcome feeds back into the agent's memory for future planning.
-

4. Folder Structure

```
hyderabad-traffic-agent/
├─ README.md
├─ pyproject.toml
├─ .env.example
├─
├─ config/
│   └─ settings.yaml          # Hyderabad bbox, ward count,
thresholds
│   └─ authority_routing.yaml  # issue type → authority mapping
│   └─ agent_prompts/
│       └─ planner_prompt.md
│       └─ citizen_intake_prompt.md
│       └─ traffic_prompt.md
│       └─ pollution_prompt.md
│       └─ energy_prompt.md
├─
├─ data/
│   └─ geo/
│       └─ ghmc_wards.geojson
│       └─ hyderabad_road_network.osm
│       └─ hmda_row_landuse.geojson  # right-of-way / land-use layers
│       └─ ghmc_drain_network.geojson
│   └─ raw/
│   └─ processed/
│   └─ vector_store/          # pgvector tables
├─
├─ src/
│   └─ auth/
│       └─ citizen_auth.py
│       └─ planner_auth.py      # issues ward_id-scoped JWT
│       └─ rbac.py
│   └─
│   └─ complaints/
│       └─ intake_api.py
│       └─ media_pipeline.py    # FFmpeg + Whisper
│       └─ classifier.py
│       └─ ward_matcher.py      # PostGIS point-in-polygon
│       └─ alert_dispatcher.py
│   └─
│   └─ agent/
│       └─ core_agent.py
│       └─ constraint_parser.py  # budget/spatial/other → hard filters
```

```
| | | └─ planner.py
| | | └─ memory.py
| | | └─ tools.py
|
| └─ rag/
| | └─ ingestion.py # Hyderabad-only scoped ingestion
| | └─ retriever.py
| | └─ vector_db.py
|
| └─ domains/
| | └─ traffic/
| | | └─ data_connector.py # HTP, GHMC, OSM, live traffic API
| | | └─ congestion_scorer.py
| | | └─ top_problems.py
| | | └─ evaluator.py
| | | └─ infra_advisor/
| | | | └─ problem_diagnoser.py
| | | | └─ rerouting_engine.py
| | | | └─ sumo_impact_checker.py
| | | | └─ signal_optimizer.py
| | | | └─ widening_flyover_advisor.py
| | | | └─ drainage_advisor.py
| | | | └─ cost_estimator.py
| | | | └─ ranker.py
| | └─ pollution/
| | | └─ data_connector.py # TSPCB, CPCB, HMWSSB, GHMC SWM
| | | └─ top_problems.py
| | | └─ traffic_linkage.py
| | └─ energy/
| | | └─ data_connector.py # TSSPDCL
| | | └─ top_problems.py
| | | └─ traffic_linkage.py
|
| └─ api/
| | └─ main.py
| | └─ routes/
| | | └─ citizen_routes.py
| | | └─ planner_routes.py
| | | └─ locality_routes.py
| | | └─ agent_routes.py
| | └─ schemas/
|
| └─ orchestration/
| | └─ scheduler.py # Airflow DAG triggers
```

```
| | | └─ event_bus.py
| |
| | └─ utils/
| |   └─ logging.py
| |   └─ geo_utils.py
|
└─ dashboard/
  └─ citizen-app/
    └─ src/
  └─ planner-app/
    └─ src/
      └─ components/
        └─ LocalitySearch.tsx
        └─ OverviewCard.tsx
        └─ DomainTabs.tsx
        └─ TopProblemsPanel.tsx
        └─ AgentConstraintChat.tsx
      └─ pages/
    └─ notebooks/                                     # SUMO scenario prototyping, model
tuning
  └─ tests/
    └─ unit/
    └─ integration/
  └─ deployment/
    └─ Dockerfile
    └─ docker-compose.yaml
    └─ k8s/
  └─ docs/
    └─ architecture.md
    └─ api_reference.md
```

5. Datasets (Hyderabad-specific, with justification)

Traffic

Source	Gives	Why this one
Hyderabad Traffic Police (HTP) / Telangana Police open data	Junction incident data, black spots	Ground-truth enforcement data; maps directly to the responsible authority
GHMC / HMDA GIS (wards, road hierarchy, right-of-way, land use)	Ward polygons, ROW width for feasibility checks	Needed for ward-matching and widening/flyover feasibility
OpenStreetMap, Hyderabad bbox (OSMnx)	Street-level road graph	Free, detailed enough for routing/simulation
Google Maps / HERE Traffic API (Hyderabad only)	Live congestion/speed	Static data alone can't give current traffic state
TSRTC route/stop data	Bus network overlay	Keeps rerouting/widening suggestions transit-aware
Historical traffic volume time series	Growth trend	Input to Prophet forecasting for widening/flyover decisions

Pollution

Source	Gives	Why
TSPCB real-time AQI stations	Official air quality	State authority — direct routing target
CPCB CAAQMS Hyderabad stations	Cross-check air data	Fills TSPCB station gaps
HMWSSB water quality reports	Water pollution snapshot	Only credible local source
GHMC Solid Waste Management data	Land pollution / dumping	Matches the land-pollution part of the overview card
GHMC stormwater drain network + IMD rainfall	Waterlogging correlation, drain sizing	Needed for the drainage-advisor sub-model

Energy

Source	Gives	Why
TSSPDCL open/consumption data	Local grid load	The actual Hyderabad utility — recommendations must route here
data.telangana.gov.in energy datasets	Historical consumption trends	Free, locality-tagged

Cost & feasibility

Source	Gives	Why
GHMC/PWD schedule of rates	Cost benchmarks (₹/km widening, ₹/lane-km flyover, ₹/km drain)	Turns recommendations into actual budget-checkable numbers
HMDA land-use / no-construction zone layers	Site feasibility	Prevents recommending infeasible sites

All ingestion is hard-filtered to the Hyderabad bounding box at query time, not just display time, so the RAG index stays precise and the agent can't pull in another city's data from a nationwide source.

6. Authority Routing Table

Issue type	Responsible authority
Traffic signal timing, junction congestion, black spots	Hyderabad Traffic Police
Road condition, widening, footpaths	GHMC Engineering (Roads) wing
Flyover construction	GHMC / HMDA (major works)
Illegal parking / encroachment	GHMC + Traffic Police (joint)
Air quality / vehicular emissions	TSPCB
Water pollution / drainage / waterlogging	HMWSSB
Land pollution / waste dumping	GHMC Solid Waste Management wing
Power load / demand response / streetlights	TSSPDCL, GHMC Electrical wing
Public transit routing	TSRTC

Stored as structured config (`config/authority_routing.yaml`) rather than hardcoded in a prompt, so it can be updated without touching agent logic.

7. Design Principles Kept Throughout

- **Traffic is structurally primary:** it's the only domain with a full recommendation engine (§1.4); pollution and energy each carry a `traffic_linkage.py` instead of duplicating full pipelines.
- **Constraints are parsed, not just prompted:** budget/spatial/other constraints are extracted into structured filters before candidate generation, so they can't be silently ignored in a long LLM response.
- **Explainable before black-box:** every model prefers a classical, auditable method first (Webster's formula, rational-method drain sizing, GIS feasibility rules) and only adds ML/RL refinement where there's enough data to trust it.
- **No auto-execution:** every output is a recommendation with a named authority — a human planner must act on it. Nothing in this design changes a real signal or closes a real road by itself.
- **Ward-scoped by default, searchable beyond:** matches how officials actually work, while still allowing cross-ward corridor issues to be investigated.